专升本 **Java** 程序设计模拟卷-卷 **6**

fifine-专升本

2026 年 3 月 2 日

fifine专升本

1 一、单项选择题深度解析（共 20 小题）

第 1 题：Java 源文件 public 类数量限制 题目：一个标准的 Java 源文件中，最多可以包含几个被 public 修饰的外部类？（ ）

A. 0 个

B. 1 个

C. 2 个

D. 不限制数量

答案：**B. 1 个**

【核心认知框架】

- 题目本质：考查 Java 编译单元（Compilation Unit）的文件结构规范与类加载机制的映射关系。
- 关键区分点：public 类名必须与文件名完全一致，决定了唯一性。
- 命题人意图：测试考生对 Java 文件系统与包机制底层规则的理解。

【选项原理深度拆解】

- **A. 0 个—错误（可以没有 public 类）**
  - 字节码铁证：

```java
// File: Test.java
class A {} class B {}
// javac Test.java 生成 A.class, B.class
```

    JVM 保障：JVM 不强制要求源文件必须有 public 类，但若有，只能有一个。
  - 陷阱：题目问“最多”，0 个是允许的最小值，非最大值。
- **B. 1 个—正确（编译单元约束）**
  - **JLS 规范**：JLS §7.6 规定，如果一个编译单元包含 public 类，其名称必须与文件名匹配。
  - 文件系统映射：操作系统文件名唯一性 → public 类唯一性。
  - 反例验证：

```java
// File: Test.java
public class A {}
public class B {} //  编译错误：类 B 是公共的，应在名为 B.java 的文件中声明
```

- **C. 2 个—错误（违反文件命名规则）**
  - 编译期检查：javac 会直接报错，无法生成字节码。
  - 类加载器：无法确定哪个类对应文件名，导致 ClassLoader 查找失败。
- **D. 不限制数量—错误（内部类除外）**

fffine专升本

– 区分概念：外部类（Top-level class）受限，内部类（Inner class）不受限。
– 字节码表现：内部类编译为 `Outer$Inner.class`，不占用主文件名。

【应背诵知识点】

1. **public** 类唯一性：一个.java 文件最多一个 public 类，且名同文件。
2. 非 **public** 类：可以有多个，但不能被包外访问。
3. 内部类：不计入外部类数量限制。

【命题趋势与实战策略】

- 考场秒杀：见 public 想文件名，文件名唯一 → 类唯一。
- 高频陷阱：混淆内部类与外部类计数。

第 2 题：byte 数据类型转换溢出 题目：执行数据类型转换代码 `byte b = (byte)130;` 后，变量 b 的值为（　）。

A. 130
B. -126
C. 编译报错
D. 运行时异常

答案：**B. -126**

【核心认知框架】

- 题目本质：考查基本数据类型强制转换时的截断机制与补码表示。
- 关键区分点：`byte` 范围 [-128, 127]，超出范围发生溢出回绕。
- 命题人意图：测试二进制位运算与补码理解。

【选项原理深度拆解】

- **A. 130** —错误（超出 **byte** 范围）
  – 范围限制：`byte` 为 8 位有符号整数，最大值 127。
  – **JVM** 行为：不会自动扩容，直接截断高位。
- **B. -126** —正确（补码截断）
  – 二进制推导：

```
130 (int) = 0000 0000 0000 0000 0000 0000 1000 0010
(byte) 截断低 8 位 = 1000 0010
补码转原码：符号位 1(负) + 数值位取反加 1 = 1111 1110 = -126
```

  – 字节码指令：`i2b` (int to byte) 执行截断。
- **C.** 编译报错—错误（显式强转合法）
  – 语法规则：窄化转换（Narrowing Primitive Conversion）需显式强转，加了 `(byte)` 即合法。
- **D.** 运行时异常—错误（位操作无异常）
  – **JVM** 规范：基本类型转换不抛出异常，仅数据丢失。

fifine专升本

【应背诵知识点】

1. **byte** 范围：-128 ~ 127。
2. 溢出公式：$Value = Original - 256$ (若 $Original > 127$)。
3. 强转规则：必须显式声明，否则编译错。

【命题趋势与实战策略】

- 考场秒杀：130 - 256 = -126。
- 高频陷阱：忘记补码符号位。

第 3 题：位运算符 » 与 »> 答案：**B.** » 是带符号右移，»> 是无符号右移 【核心认知框架】考查移位运算符对符号位的处理方式。【选项原理深度拆解】

- **A.** 完全无区别：错误，负数时结果不同。
- **B.** 正确：>> 补符号位，>>> 补 0。
- **C.** 反了：错误，定义相反。
- **D.** 浮点数：错误，移位仅用于整数。

【应背诵知识点】 >> 保留符号，>>> 逻辑右移。

第 4 题：二维数组长度 答案：**A. arr.length**【核心认知框架】考查数组属性访问。【选项原理深度拆解】

- **A.** 正确：数组对象有 `length` 属性。
- **B.** 列数：`arr[0].length` 获取列。
- **C. size()**：集合方法，数组无。
- **D. length()**：String 方法，数组无。

第 5 题：访问修饰符权限 答案：**D. private** 【核心认知框架】考查封装性级别。【选项原理深度拆解】

- **A. public**：最宽。
- **B. protected**：子类 + 同包。
- **C.** 默认：同包。
- **D. private**：仅本类，最严格。

第 6 题：方法重载 答案：**A.** 方法名相同，参数列表不同【核心认知框架】考查 Overload 定义。【选项原理深度拆解】

- **A.** 正确：编译期多态。
- **B.** 返回值：不足以区分重载。
- **C.** 修饰符：不足以区分。
- **D.** 子类覆盖：那是 Override。

tifine专升本

第 7 题：多态方法调用 答案：**B.** 子类重写后的方法 【核心认知框架】考查动态绑定。【选项原理深度拆解】

- **A.** 父类：静态绑定才如此。
- **B.** 正确：运行时根据对象实际类型决定。
- **C/D.** 异常：多态是合法特性。

第 8 题：static 关键字 答案：**A.** 静态方法内部可以直接调用同一个类中的非静态方法 【核心认知框架】考查静态上下文限制。【选项原理深度拆解】

- **A.** 错误（即正确答案）：静态无 `this`，不能调非静态。
- **B.** 类名调用：正确。
- **C.** 共享：正确。
- **D.** 加载执行：正确。

【字节码铁证】 `invokestatic` 无法访问实例方法 `invokevirtual` without instance。

第 9 题：接口默认方法 答案：**A. default** 或 **static** 【核心认知框架】考查 JDK8 接口特性。【选项原理深度拆解】

- **A.** 正确：JDK8 新增。
- **B. abstract**：默认就是 abstract。
- **C. private**：JDK9 才支持私有方法。
- **D. final**：接口方法默认抽象，不能 final（除非 default）。

第 10 题：局部内部类变量 答案：**B. final** 【核心认知框架】考查闭包实现机制。【选项原理深度拆解】

- **A. static**：局部类不能 static。
- **B.** 正确：保证生命周期一致，JDK8 隐式 effective final。
- **C/D.** 无关：修饰符不匹配。

第 11 题：throws 关键字 答案：**C.** 声明该方法执行过程中可能抛出的异常类型 【核心认知框架】考查异常声明机制。【选项原理深度拆解】

- **A.** 抛出：那是 `throw`。
- **B.** 捕获：那是 `catch`。
- **C.** 正确：告知调用者。
- **D.** 停止：那是 `System.exit`。

fifine专升本

第 12 题：String 不可变性 答案：**B.** 产生了一个全新的字符串对象**"Java SE"**，变量 **s** 指向新对象 【核心认知框架】考查 String 池与对象创建。【选项原理深度拆解】

- **A.** 修改：String 不可变。
- **B.** 正确：+ 创建新对象。
- **C.** 编译错：支持 + 重载。
- **D.** 异常：逻辑正常。

第 13 题：自动装箱 答案：**A.** 将基本数据类型自动转换为对应的包装类对象 【核心认知框架】考查泛型与集合支持。【选项原理深度拆解】

- **A.** 正确：`int` → `Integer`。
- **B.** 拆箱：反向过程。
- **C.** 解析：`valueOf`。
- **D.** 序列化：IO 流。

第 14 题：LinkedHashMap 答案：**C. LinkedHashMap** 【核心认知框架】考查集合顺序特性。【选项原理深度拆解】

- **A. HashMap**：无序。
- **B. TreeMap**：排序。
- **C.** 正确：链表维护插入顺序。
- **D. Hashtable**：老旧无序。

第 15 题：泛型通配符 答案：**A.** 集合中的元素类型必须是 **Number** 或其子类 【核心认知框架】考查 PECS 原则。【选项原理深度拆解】

- **A.** 正确：`extends` 上界。
- **B.** 父类：那是 `super`。
- **C.** 只能 **Number**：那是 `List<Number>`。
- **D.** 任何：那是 `<?>`。

第 16 题：处理流 答案：**B. BufferedReader** 【核心认知框架】考查节点流与处理流区别。【选项原理深度拆解】

- **A/C/D.** 文件流：直接连数据源，节点流。
- **B.** 正确：包装其他流，提供缓冲。

第 17 题：wait() 锁机制 答案：**A.** 对象的锁（监视器）【核心认知框架】考查线程协作前提。【选项原理深度拆解】

- **A.** 正确：必须 synchronized 块内。

fifine专升本

- **B/C/D.** 无关：非锁资源。

【**JVM** 规范】 `IllegalMonitorStateException` 若未持有锁。

第 18 题：事件适配器 答案：**B.** 避免必须实现接口中的所有抽象方法，只需重写需要的方法 【核心认知框架】考查设计模式（适配器）。【选项原理深度拆解】

- **A.** 性能：无显著提升。
- **B.** 正确：简化代码。
- **C/D.** 功能：无关。

第 19 题：JDBC 关闭顺序 答案：**B. ResultSet → Statement → Connection** 【核心认知框架】考查资源依赖关系。【选项原理深度拆解】

- **A/C.** 顺序错：依赖方先关。
- **B.** 正确：后创建的先关闭。
-
- **D.** 无关：资源泄露风险。

第 20 题：volatile 关键字 答案：**C. volatile** 【核心认知框架】考查 JMM 可见性。【选项原理深度拆解】

- **A. synchronized**：保证原子性 + 可见性。
- **B. transient**：序列化忽略。
- **C.** 正确：仅可见性，不保证原子性。
- **D. final**：不可变。

2 二、判断题深度解析（共 10 小题）

第 1 题：main 方法参数名 题目： Java 入口方法 `main` 的参数名必须叫 `args`，否则程序无法启动。答案：× 【核心认知框架】考查 JVM 启动规范。【深度解析】

- **JVM** 规范：JVM 查找 `public static void main(String[])` 签名，参数名无关。
- 字节码：方法描述符只看类型，不看变量名。
- 验证：`main(String[] x)` 同样可运行。

第 2 题：构造方法 return 题目：构造方法没有返回值类型，因此在构造方法体中绝对不能使用 `return;` 语句。答案：× 【核心认知框架】考查构造方法控制流。【深度解析】

- 语法规则：可以使用 `return;` 提前结束构造，但不能返回值。

fifine专升本

- 字节码：`return` 指令用于 void 方法。

第 3 题：局部变量初始化 题目：Java 中局部变量在使用之前必须进行显式的初始化，否则编译器会报错。答案：*checkmark* 【核心认知框架】考查数据流分析。【深度解析】
- 编译器检查：局部变量无默认值，必须赋值后使用。
- 对比：成员变量有默认值。

第 4 题：抽象类抽象方法 题目：抽象类中必须至少包含一个抽象方法，否则无法被声明为抽象类。答案：× 【核心认知框架】考查抽象类定义。【深度解析】
- 设计意图：抽象类可仅为了禁止实例化，无需抽象方法。
- 示例：`abstract class Base { }` 合法。

第 5 题：hashCode 与 equals 题目：两个对象的 `hashCode()` 返回值相同，则它们的 `equals()` 方法比较也必定返回 `true`。答案：× 【核心认知框架】考查哈希冲突。【深度解析】
- 哈希契约：equals 真 → hashCode 同；反之不成立。
- 冲突：不同对象可能哈希值相同。

第 6 题：受检异常 题目：受检异常（Checked Exception）在程序编写时必须通过 `try-catch` 捕获或通过 `throws` 声明抛出。答案：*checkmark* 【核心认知框架】考查异常处理机制。【深度解析】
- 编译器强制：非 RuntimeException 必须处理。

第 7 题：File 类写入 题目：`java.io.File` 类的对象可以直接调用方法向文件中写入字符串内容。答案：× 【核心认知框架】考查 File 类功能。【深度解析】
- **File** 职责：仅管理路径/元数据，IO 操作需流（Stream/Writer）。

第 8 题：StringBuffer 线程安全 题目：`StringBuffer` 类的很多方法使用 `synchronized` 进行了修饰，因此它是线程安全的。答案：*checkmark* 【核心认知框架】考查并发工具类。【深度解析】
- 实现：方法级锁，保证原子性。
- 对比：StringBuilder 非线程安全。

fifine专升本

第 9 题：sleep 锁释放 题目：当线程调用 `Thread.sleep()` 方法进入睡眠时，该线程会释放它所持有的对象锁。答案：× 【核心认知框架】考查线程状态与锁。【深度解析】

- **sleep** 行为：暂停执行，不释放锁。
- **wait** 行为：释放锁。

第 10 题：PreparedStatement 题目：`PreparedStatement` 不仅能够有效防止 SQL 注入攻击，并且在执行结构相同的 SQL 时效率更高。答案：*checkmark* 【核心认知框架】考查 JDBC 安全与性能。【深度解析】

- 预编译：数据库缓存执行计划，防止注入。

3 三、填空题深度解析（共 5 小题）

第 1 题：Java 运行机制 题目：Java 程序的运行机制分为两个主要阶段，首先是 编译 阶段将源文件转为字节码，然后是 解释（或运行） 阶段由 JVM 执行字节码。【核心认知框架】考查 Java 跨平台原理。【深度解析】

- 编译：`javac` 生成.class。
- 运行：JVM 加载、验证、执行。
- **JIT**：现代 JVM 包含即时编译，混合执行。

第 2 题：构造方法与访问修饰符 题目：在定义类时，如果不显式编写任何构造方法，编译器会自动提供一个 无参（或默认） 的构造方法；如果类的成员变量被 private 修饰，则它只能在当前类内部被访问。【核心认知框架】考查默认行为与封装。【深度解析】

- 默认构造：仅当无显式构造时生成。
- **private**：最严格访问控制。

第 3 题：super 关键字 题目：`super` 关键字除了能在子类中调用父类的构造方法，还可以用于访问父类被隐藏的 成员变量（或属性） 和被子类重写的 成员方法（或方法）。【核心认知框架】考查继承访问机制。【深度解析】

- 隐藏：变量同名。
- 重写：方法同签名。
- 字节码：`invokespecial` 调用父类方法。

fffine专升本

第 4 题：IO 转换流 题目：在 Java I/O 包中，用于将字节输入流转换为字符输入流的转换流是 InputStreamReader；为了提高字符流的读取效率，通常会外层包装一个 BufferedReader 流。【核心认知框架】考查 IO 流体系。【深度解析】

- 转换：字节 → 字符（解码）。
- 缓冲：减少 IO 次数，提升性能。

第 5 题：线程同步 题目：线程同步机制中，使用 synchronized 关键字可以对方法或代码块进行加锁控制；若要唤醒在对象监视器上等待的某一个线程，需调用该对象的 notify() 方法。【核心认知框架】考查并发协作。【深度解析】

- 锁：内置监视器锁。
- 唤醒：`notify` 随机唤醒一个，`notifyAll` 唤醒所有。

4 四、阅读程序，写出运行结果深度解析（共 5 小题）

第 1 题：循环与 break 标签 代码：双重循环，`i*j > 4` 时 `break outer`。答案：`Count = 5` 【核心认知框架】考查带标签的 break 语句。【执行流程深度拆解】

- **i=1**: j=1,2,3 (count=3), 1*3<=4.
- **i=2**: j=1 (count=4), j=2 (2*2=4, count=5), j=3 (2*3=6>4, break outer).
- **i=3**: 不执行。
- 总计：5 次。

【命题陷阱】容易忽略 break outer 跳出的是外层循环。

第 2 题：构造方法调用链 代码：`new ClassB(1)` 调用 `this()` 再调父类构造。答案：`ABC` 【核心认知框架】考查构造方法执行顺序。【执行流程深度拆解】

- **Step 1**: `ClassB(int)` 调用 `this()`。
- **Step 2**: `ClassB()` 隐式调用 `super()`。
- **Step 3**: `ClassA()` 打印"A"。
- **Step 4**: `ClassB()` 继续打印"B"。
- **Step 5**: `ClassB(int)` 继续打印"C"。

【**JVM** 规范】构造链必须第一条语句调用其他构造或 super。

第 3 题：try-catch-finally 返回值 代码：try 除零异常，catch 返回，finally 打印。答案：`FinallyBlock CatchBlock` 【核心认知框架】考查 finally 执行时机。【执行流程深度拆解】

- 异常：10/0 抛出 ArithmeticException。

ffine专升本

- **catch**: 准备返回"CatchBlock"。
- **finally**: 优先执行，打印"FinallyBlock "。
- **return**: finally 结束后返回 catch 的值。

【命题陷阱】 finally 中有 return 会覆盖 catch 的 return，本题没有。

第 4 题：String 常量池 代码： `s1`，`s2` 字面量，`s3` new 对象。答案：`true` `false` 【核心认知框架】考查字符串内存分布。【执行流程深度拆解】

- **s1==s2**: 指向常量池同一对象 → true。
- **s1==s3**: `new` 在堆中新建对象 → false。

【字节码铁证】 `ldc` (常量池) vs `new` + `invokespecial` (堆)。

第 5 题：引用传递与对象修改 代码：修改 `p.x`，重新赋值 `p`。答案：`15`，`5` 【核心认知框架】考查引用传递本质。【执行流程深度拆解】

- **p.x += 10**: 修改堆中对象，`pt.x` 变为 15。
- **p = new Point()**: 形参 p 指向新对象，不影响实参 pt。
- **pt.y**: 保持原值 5。

【命题陷阱】误以为 `p = new` 会改变外部引用。

5 五、编程题深度解析与参考实现（共 2 小题）

第 1 题：类的设计与封装（Book 类）题目要求：私有属性、构造方法、Getter/Setter（价格校验）、display 方法、测试类。【核心认知框架】考查 JavaBean 规范与封装性。【设计深度解析】

- 封装：`private` 属性防止外部直接修改。
- 校验逻辑：在 Setter 中集中处理数据合法性，构造器复用 Setter。
- 测试：验证边界条件（负数价格）。

【参考代码实现】

```java
class Book {
    private String title;
    private String author;
    private double price;

    public Book(String title, String author, double price) {
        this.title = title;
        this.author = author;
        setPrice(price); // 复用校验逻辑
```

fitme专升本

```java
    }
    // Getter/Setter 省略...
    public void setPrice(double price) {
        this.price = (price < 0) ? 0.0 : price;
    }
    public void display() {
        System.out.println("书名: " + title + ", 作者: " + author + ", 价格: " + price);
    }
}
public class TestBook {
    public static void main(String[] args) {
        Book book = new Book("Java 进阶", "李四", -20.0);
        book.display(); // 输出价格应为 0.0
    }
}
```

【评分关键点】属性私有化 (3 分)、构造器 (3 分)、Setter 校验 (5 分)、测试覆盖 (5 分)。

第 2 题：继承、抽象类与多态应用（员工工资）题目要求：抽象父类 Employee、子类 FullTime/PartTime、多态测试。【核心认知框架】考查 OOP 三大特性（继承、封装、多态）。【设计深度解析】

- 抽象类：定义通用接口 calculateSalary，强制子类实现。
- 多态：父类引用指向子类对象，统一调用方法。
- 扩展性：新增员工类型只需新增子类，无需修改测试代码（开闭原则）。

【参考代码实现】

```java
abstract class Employee {
    private String name, id;
    public Employee(String name, String id) { this.name = name; this.id = id; }
    public String getName() { return name; }
    public abstract double calculateSalary();
}
class FullTimeEmployee extends Employee {
    private double baseSalary;
    public FullTimeEmployee(String name, String id, double baseSalary) {
        super(name, id); this.baseSalary = baseSalary;
    }
```

ffine专升本

```java
    public double calculateSalary() { return baseSalary; }
}
// PartTimeEmployee 类似实现...
public class TestEmployee {
    public static void main(String[] args) {
        Employee e1 = new FullTimeEmployee("王五", "F001", 8000.0);
        System.out.println(e1.getName() + " 工资: " + e1.calculateSalary
            ());
    }
}
```

【评分关键点】抽象类定义 (5 分)、子类实现 (5 分)、多态调用 (5 分)、代码规范 (5 分)。

fifine专升本